山东建筑大学 计算机学院 实验报告

班级：软件 212　姓名：孙帅康　学　号：202111110377　实验成绩：_____

课程：计算机网络　同组者：_____　实验日期：2023/5/19

实验二　多线程 Web 服务器的设计与实现

一、实验目的及任务

1、实验目的

熟悉简单网络的搭建与基本配置；

熟悉 socket、多线程编程；

熟悉 JDK 编程工具的基本使用；

熟悉 HTTP 协议；

熟悉 Web 服务器的基本工作原理和配置。

2、实验任务

以 JDK 为开发工具，利用 Socket 通信机制实现一个多线程的 WEB 服务器。服务器具有以下功能：

- 能够并行服务于多个请求。
- 对于每个请求，显示接收到的 HTTP 请求消息的内容，并产生适当的响应(若找到用户请求对象，则返回该对象；否则，发送一个包含适当提示信息的响应消息，从而可以在浏览器窗口中显示差错信息。)

二、实验性质及学时

设计性实验，4 学时

三、实验环境

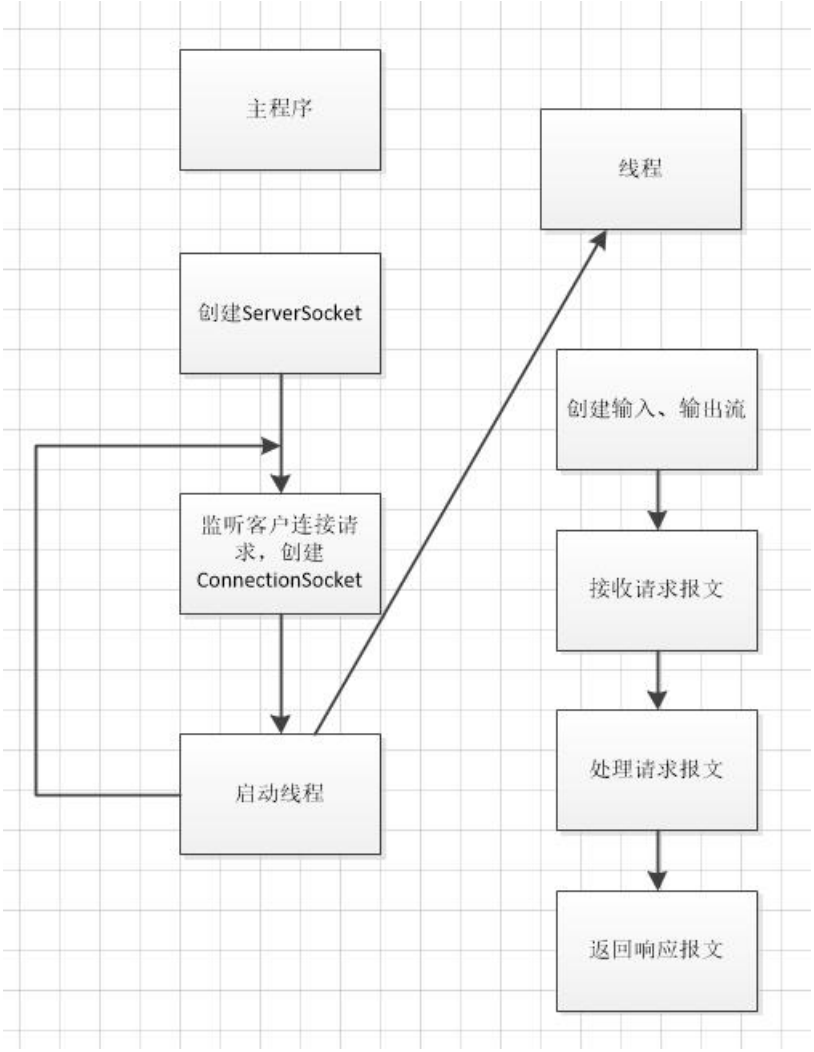
操作系统：Windows

开发工具：JDK 或其他开发工具

实验设备：交换机(一台)、计算机(两台)、网线若干

四、实验报告内容（实验题目及答案，其中答案部分标红）

1、写出实现“一、实验目的及任务”中“2、实验任务”所述功能的 Web 服务器程序的流程图。



2、写出实现“一、实验目的及任务”中“2、实验任务”所述功能的 Web 服务器程序的主要代码。

（1）RequestExecute.java 请求处理线程类

```
import java.io.*;
import java.net.Socket;
    //构造方法: 迫使 Server 调用此类的时候必须传一个 socket 进来
    public RequestExecute(Socket socket){
        this.socket = socket;
    }
        try{
            //从 socket 中获取字节输入流
            in = socket.getInputStream();
            //将自己而输入流包装成字符流
            inReader = new InputStreamReader(in);
            //将字符流包装成缓冲流
            bReader = new BufferedReader(inReader);
```

```
//从 socket 中获取字节输出流
out = socket.getOutputStream();
//将字节输出流包装成字符流
pw = new PrintWriter(out);

//循环的从字节流中获取字符
String line = null;

//记录正在解析请求报文的哪一行
int linNum = 0;
//存储请求路径
String reqPath = "";
//存储端口号
String host = "";

while ((line=bReader.readLine())!=null){

    //逐行输出 HTTP 请求报文的每一行
    System.out.println(line);
    linNum++;
    //逐行分析 HTTP 请求报文的每一行
    if(linNum==1){//表名是这个 HTTP 请求报文的第一行(第一行包含了请求内容，徐单独分析)
        //使用空格分割字符串，为了得到请求路径
        String[] infos = line.split(" ");//      GET /test HTTP/1.1    第一行内容
        if(infos.length > 2){     //说明解析成功了
            //取出请求路径
            reqPath = infos[1];
        }else { //解析失败
            throw new RuntimeException("Failed to analyse the Request: "+line);
        }
    }else {
        //解析其他行,主要目的是取出 host 内容，得到客户端请求报文使用的端口号
        String[] infos = line.split(": ");//第二行: Host: localhost:8088
        if(infos.length == 2){
            if(infos[0].equals("Host"))
                host = infos[1];
        }
    }
//当请求报文解析成功了
if(!reqPath.equals("")) {

    //在控制台输出请求信息:我们需要解析的端口号和请求信息
    System.out.println("the request that execute: http://" + host + reqPath);
```

```
//根据请求响应客户端 如果是/后空就返回一个欢迎页面(首页) 如果是指定的静态页面就返回一个静态文件
if(reqPath.equals("/")){
    pw.println("HTTP/1.1 200 OK");//输出响应报文的头部行的第一行
    pw.println("Content-Type: text/html;charset=utf-8");//第二行
    pw.println();//表示响应头结束，开始相应内容
    File file = new File("E://RangerServerStatic//index.html");
    Response200(out,file.getAbsolutePath(),"html");//主页：响应读取静态资源中的首页
    pw.flush();//刷新
    System.out.println();

}else{

    //查找对应的资源   /index.html /app/Gandalf.jpg....
    String ext = reqPath.substring(reqPath.lastIndexOf(".")+1);//取出后缀并把"."去掉
    reqPath = reqPath.substring(1); //   去掉请求前面的"/"，方便下面的是否是在子目录下的判断
    //判断请求是根目录下还是其他子目录下
    if(reqPath.contains("/")){   //如果前面取出斜杠之后还是有斜杠，只能说明是在子目录下
        File file = new File(Data.resourcesPath+"/"+reqPath);
        //判断文件是否存在
        if(file.isFile()&& file.exists()){
            //将文件相应到客户端
            Response200(out,file.getAbsolutePath(),ext);
        }else {
            //相应一个自定义的 404 页面
            Response404(out);
        }

    }else {        //说明是在根目录下
        //判断这个资源是否存在
        //获取根目录下所有的资源名称
        File root = new File(Data.resourcesPath);
        if(root.isDirectory()){        //如果是个目录的话
            File[] list = root.listFiles(); //获取根目录下所有的文件
            boolean isExist = false;
            for(File file:list){
                if(file.isFile() && file.getName().equals(reqPath)){
                    //说明文件存在
                    isExist = true;
                    break;
                }
            }
            if(isExist){      //文件存在
```

```
                    Response200(out, Data.resourcesPath+"/"+reqPath, ext);
                }else {             //文件不存在
                    Response404(out);
                }
            }else {
                throw new RuntimeException("Static Resources Directory not Exist! :"+Data.resourcesPath);
            }
        }
    }
}

}catch (IOException e){
    e.printStackTrace();
    throw new RuntimeException(e);
}finally {
    try{

        //关闭流，保护线程
        if(in!=null)
            in.close();
        if(inReader!=null)
            inReader.close();
        if(bReader!=null)
            bReader.close();
        if(pw!=null)
            pw.close();
        if(out!=null)
            out.close();

    }catch (IOException ex){
        ex.printStackTrace();
    }
}
}

//能找得到对应文件的时候
private void Response200(OutputStream out,String filePath,String ext){
    PrintWriter pw = null;
    //准备输入流读取磁盘上的文件
    InputStream in = null;
    InputStreamReader reader = null;
    BufferedReader bufferedReader = null;
    try {

        //图片要使用字节流输出
        if(ext.equals("jpg") || ext.equals("png")||ext.equals("gif")){
```

```
out.write("HTTP/1.1 200 OK\r\n".getBytes());//输出响应行
switch (ext) {
    case "jpg":
        out.write("Content-Type: image/jpg\r\n".getBytes());
        break;
    case "png":
        out.write("Content-Type: image/png\r\n".getBytes());
        break;
    case "gif":
        out.write("Content-Type: image/gif\r\n".getBytes());
        break;
}
//输出一个空行
out.write("\r\n".getBytes());//输出空行，表示响应头结束
//利用自己而输入流读取文件内容，并且输出到输入流中
//创建输入流
in = new FileInputStream(filePath);
int len = -1;
byte [] buff = new byte[1024];
while((len = in.read(buff))!=-1){
    out.write(buff,0,len);
    out.flush();
}
//若请求文件是以下网页文档则需要文本输出
}else if(ext.equals("html") || ext.equals("js") || ext.equals("css") || ext.equals("json")||ext.equals("txt")){
pw = new PrintWriter(out);
pw.println("HTTP/1.1 200 OK");//输出响应行
switch (ext) {
    case "html":
        pw.println("Content-Type: text/html;charset=utf-8");
        break;
    case "js":
        pw.println("Content-Type: application/x-javascript");
        break;
    case "css":
        pw.println("Content-Type: text/css");
        break;
    case "json":        //json 理论上不适于静态资源：可以优化
        pw.println("Content-Type: application/json;charset=utf-8");
        break;
    case "txt":
        pw.println("Content-Type: text/html");
        break;
}
//输出空行，表示响应头结束
pw.println();
```

```
                //初始化输入流
                in = new FileInputStream(filePath);
                reader = new InputStreamReader(in);
                bufferedReader = new BufferedReader(reader);
                //写出数据
                String line = null;
                while((line = bufferedReader.readLine())!=null){
                    pw.println(line);
                    pw.flush();
                }
            }else{
                Response404(out);
            }
        }catch (Exception e){
            e.printStackTrace();
        }finally {
            try{
                if(pw!=null)
                    pw.close();
            }catch (Exception ex){
                ex.printStackTrace();
            }
        }
    }

    //找不到对应文件的时候
    private void Response404(OutputStream out){
        PrintWriter pw = null;
        try {
            pw = new PrintWriter(out);
            pw.println("HTTP/1.1 404");//输出响应行
            pw.println("Content-Type: text/html;charset=utf-8");
            //输出空行
            pw.println();//表示响应头结束，开始响应内容
//              pw.write("<h2>welcome to Ranger-Server 1.0</h2>");
            pw.write("<h2>404NOTFOUND</h2>");
            pw.flush();
            System.out.println("Response welcome!");
        }catch (Exception e){
            e.printStackTrace();
        }finally {
            try{
                if(pw!=null)
                    pw.close();
            }catch (Exception ex){
                ex.printStackTrace();
            }
```

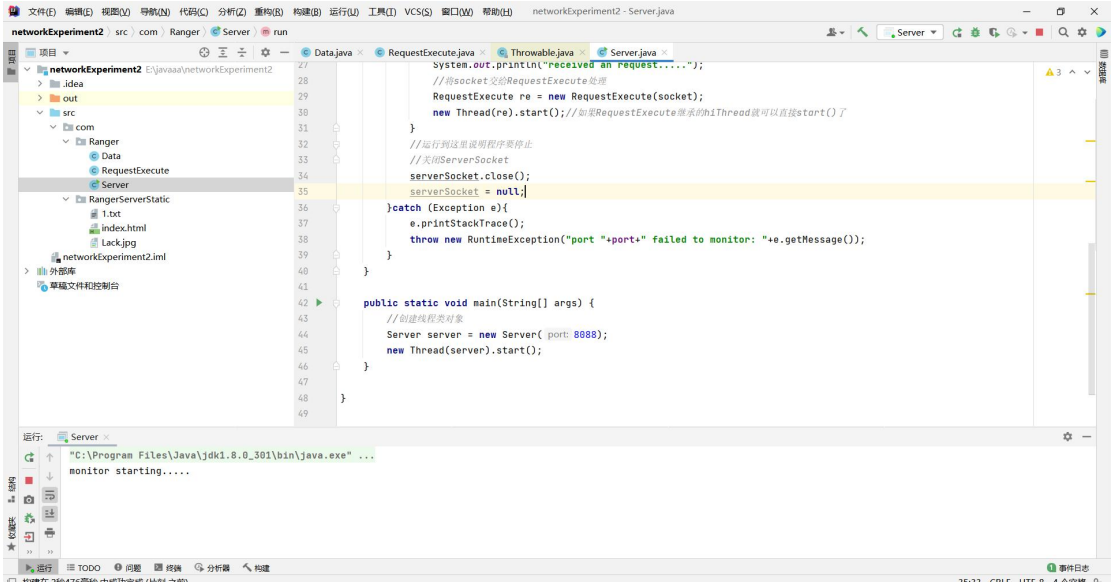
```
        }
    }

}
```

（2）Server.java

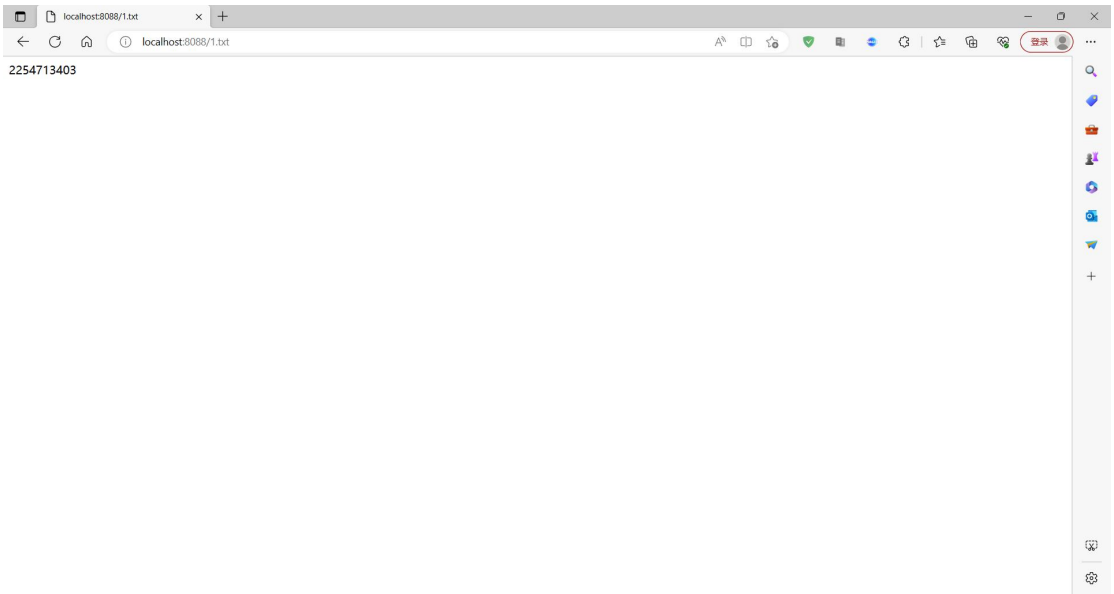
```
import java.net.ServerSocket;
import java.net.Socket;

//专门用来做服务监听
public class Server implements Runnable {

    //监听的端口
    private int port;
    public Server(int port){
        this.port = port;
    }

    public void run(){
        //线程的主程序
        //声明 ServerSocket 对象
        ServerSocket serverSocket = null;
        try{
            //创建 SeverSocket 对象
            //Socket 套接字创建之后，就可以获取监听客户端发来的 HTTP 请求报文
            serverSocket = new ServerSocket(port);
            //开始监听
            System.out.println("monitor starting.....");
            while (Data.isRun){
                Socket socket = serverSocket.accept();
                System.out.println("received an request.....");
                //将 socket 交给 RequestExecute 处理
                RequestExecute re = new RequestExecute(socket);
                new Thread(re).start();//如果 RequestExecute 继承的 hiThread 就可以直接
start()了
            }
            //运行到这里说明程序要停止
            //关闭 ServerSocket
            serverSocket.close();
            serverSocket = null;
        }catch (Exception e){
            e.printStackTrace();
            throw new RuntimeException("port "+port+" failed to monitor: "+e.getMessage());
        }
    }

    public static void main(String[] args) {
        //创建线程类对象
```

```
        Server server = new Server(8088);
        new Thread(server).start();
    }

}
```

（3）测试结果：



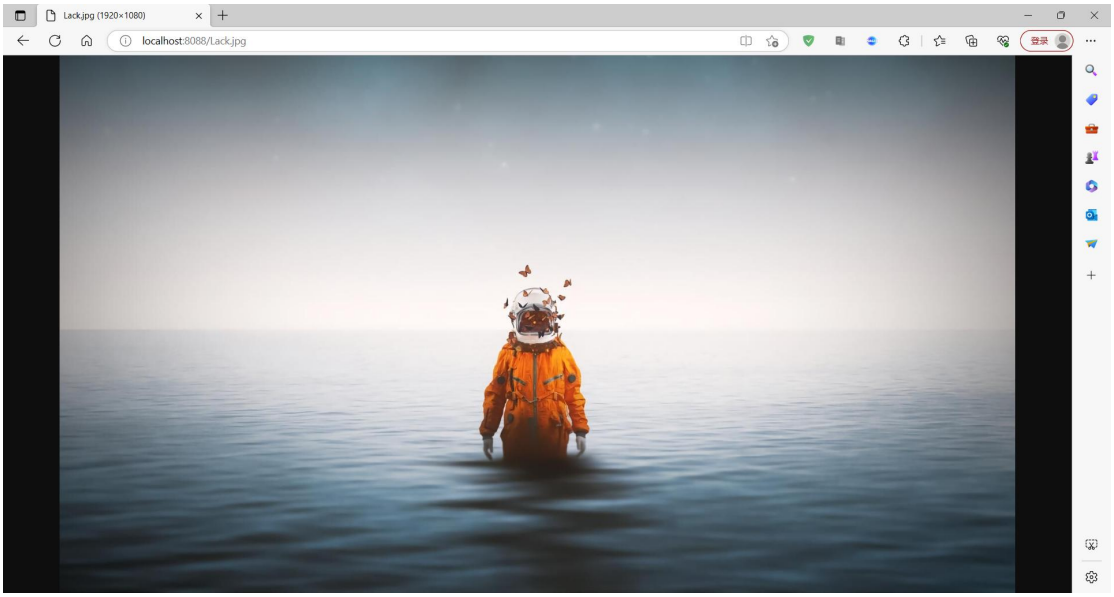
（4）访问 1.txt 文件




```
GET /1.txt HTTP/1.1
Host: localhost:8088
Connection: keep-alive
sec-ch-ua: "Microsoft Edge";v="113", "Chromium";v="113", "Not-A.Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.57
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
```

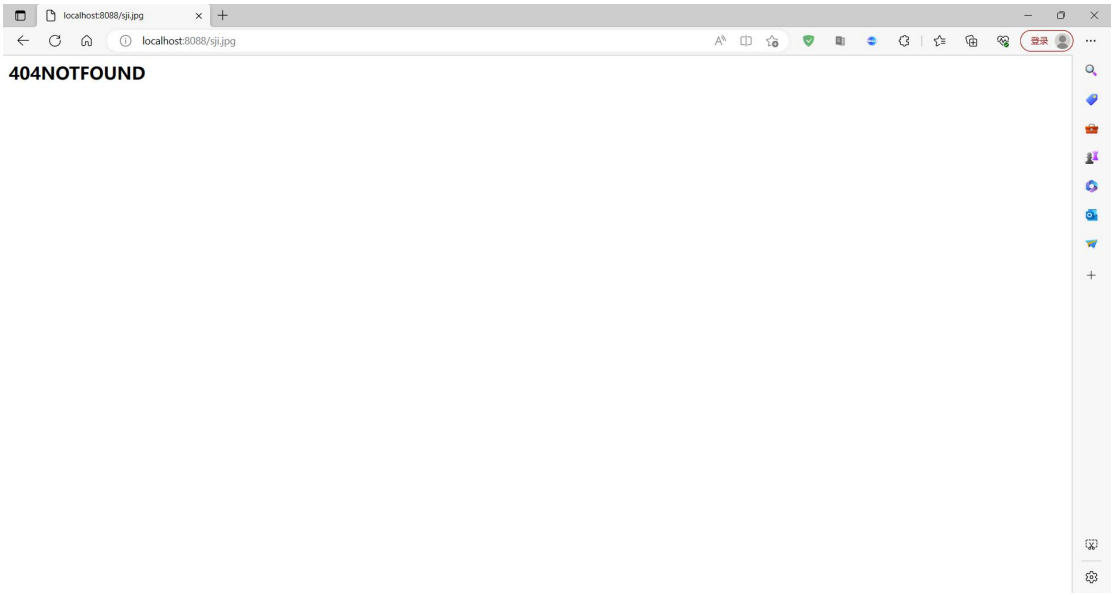
```
the request that execute: http://localhost:8088/1.txt
GET /favicon.ico HTTP/1.1
Host: localhost:8088
Connection: keep-alive
sec-ch-ua: "Microsoft Edge";v="113", "Chromium";v="113", "Not-A.Brand";v="24"
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.57
sec-ch-ua-platform: "Windows"
Accept: image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8088/1.txt
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
```

（5）访问已存在的照片




```
GET /Lack.jpg HTTP/1.1
Host: localhost:8088
Connection: keep-alive
sec-ch-ua: "Microsoft Edge";v="113", "Chromium";v="113", "Not-A.Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.57
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

the request that execute: http://localhost:8088/Lack.jpg
GET /favicon.ico HTTP/1.1
Host: localhost:8088
Connection: keep-alive
sec-ch-ua: "Microsoft Edge";v="113", "Chromium";v="113", "Not-A.Brand";v="24"
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.57
sec-ch-ua-platform: "Windows"
Accept: image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8088/Lack.jpg
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
```

（6）访问不存在的图片文件




```
GET /sji.jpg HTTP/1.1
Host: localhost:8088
Connection: keep-alive
sec-ch-ua: "Microsoft Edge";v="113", "Chromium";v="113", "Not-A.Brand";v="24"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.57
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

the request that execute: http://localhost:8088/sji.jpg
Response welcome!
GET /favicon.ico HTTP/1.1
Host: localhost:8088
Connection: keep-alive
sec-ch-ua: "Microsoft Edge";v="113", "Chromium";v="113", "Not-A.Brand";v="24"
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.57
sec-ch-ua-platform: "Windows"
Accept: image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
Referer: http://localhost:8088/sji.jpg
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

the request that execute: http://localhost:8088/favicon.ico
```

3、总结程序设计、调试中出现的问题及其解决办法。

（1）在访问本地文件时，要设置路径然后存放图片文件、文本文件等。

（2）构造方法: 迫使 Server 调用此类的时候必须传一个 socket 进来

（3）在写文件输出流时，图片用字节流输出，文本文件以文本流输出，并且要考虑多种情况，例如文件不存在时。

4、实验心得。

学习了 Runnable 接口和覆盖方法 run()，并且 Runnable 接口没有任何对线程的支持，还须创建 Thread 类的实例，通过 Thread 类的构造函数 public Thread(Runnable target)来实现，Thread 类最重要的方法是 run() ，被 Thread 的方法 start()所调用。